µParametrization Derivation 1: Fully Connected Neural Network with 3 layers (Non-exhaustive version)

Yassine Kaddami

Abstract

In the Tensor Programs series (1; 2; 3; 4; 5), Greg Yang introduced the µ-Parametrization which enables the stability of optimal hyperparameters even as model's width changes. This property allows a new hyperparameter tuning paradigm that reduces massively its associated costs. Furthermore, this parametrization is claimed to be the unique parametrization that enables hidden layers to update as much as possible, during the training process, without causing activations to blow-up. Since recent works to demonstrate these properties are very complex and rely on many assumptions, the core ideas behind the µ-parametrization and the link between its different properties are very difficult to cease. In this article, we propose a clearer and more rigorous mathematical demonstration where we show that the µ-parametrization is the unique solution of a linear optimization problem derived from studying the limit behavior, as width tends to infinity, of the output and weights of any neural network.

Contents

1 Introduction

The rise of large language models has brought about many challenges. These models, like GPT-3 with its massive 175 billion parameters, demand immense computational resources. Training such models takes weeks or even months and consumes vast amounts of energy. Hyperparameters are crucial for the network's performance, yet finding the best ones for large models is daunting. Researchers are exploring scaling techniques and hyperparameter optimization methods to tackle this challenge and make progress in handling these colossal neural networks.
One of the interesting research works on this subject is the proposition of new hyperparameter tuning paradigm, in the Tensor Programs of Greg Yang, enabled by interesting properties of the µ-parametrization. However, the work to define and demonstrate mathematically these properties is very complex and relies on many assumptions and intuitions. Therefore, we aim in this article to present a clear mathematical explanation of the properties of the µ-parametrization, and to question its unicity in regards to these properties.

Definition 1: Parametrization

In this work we define a parametrization as the rule of how to scale hyperparameters when the widths of a neural network change, but note that it does not necessarily prescribes how to set the hyperparameters for any specific width.
Example : Let's say we set at the beginning the value of a hyperparameter to be h_0 at width n_0. A parametrization is the rule giving the value of this hyperparameter for any other width: Lets say we choose now a width n, the new value h of our hyperparameter, according to our parametrization rule, will be: $h = \frac{h_0 \times n_0^p}{n^p}$ where p is a constant set by our parametrization rule.
Remark : The choice of the starting width and the starting value of any hyperparameter isn't relevant for our demonstrations. That's why, for simplification, we will always choose h_0 and n_0 such that $h_0 \times n_0^p = 1$ for any hyperparameter and any parametrization.

The interesting properties of the µ-parametrization :

Thera are two very interesting properties of the µ-parametrization:

1. The optimal hyperparameters are stable even as model's width changes
2. The hidden layers update as much as possible during the training process when the first property is satisfied

These two properties are very interesting for two main reasons:

– A new hyperparameter tuning paradigm is enabled by the first property: we can search for the optimal hyperparameters for a down-scaled model and then by scaling the model-up we can transfer these obtained hyperparameters using the µ-parametrization and they will

still remain optimal for the scaled-up model. This new paradigm will reduce considerably the hyperparameters tuning costs.

– Hidden Layers updating as much as possible is a very useful property in finetuning and transfer learning tasks. In fact, in both we want the hidden layers of our pretrained model to have learned features as much as possible.

Problem Statement :

Are these properties unique to the µ-parametrization? How can we mathematically prove that these properties are consistently maintained?
In this work, we will tackle these two questions.

Plan :

1. Necessary and sufficient conditions to derive the µ-Parametrization
2. µ-Parametrization Derivation: linear fully connected neural network with three layers
3. µ-Parametrization Derivation: MLP with three layers
4. µ-Parametrization Derivation: MLP with multiple layers
5. Explanation of stability of optimal hyperparameters trough width
6. Difference with Standard Parametrization

2 Necessary and sufficient conditions to derive the µ-Parametrization

The µ-parametrization is the unique parametrization respecting the three following conditions:

– Condition 1: In every forward pass during training, the Neural Network Output converges in probability towards a constant when the number of weigths tends to infinity.

– Condition 2: this constant depends only on the training data (training inputs and labels), and is not always null (except for the 1^{st} forward pass).

– Condition 3: Maximum update of hidden layers.

2.1 Implications of conditions 1 and 2

For large widths, these conditions guarantee that when the width of a neural network changes from n to n', and a hyperparameter h is scaled to h' according to the parametrization, the value of the output O given an input x remains unchanged. Mathematically, it can be expressed as $O_{n'}(h', x) = O_n(h, x)$, indicating that the output retains its value across different network widths when hyperparameters are appropriately scaled.

That can explain partly how with the µ-parametrization the optimal hyperparameters are stable for large models even as model's width changes. In fact, since the output remains unchanged as width changes under the µ-parametrization scaling, the loss function is also unchanged, and therefore, the minimum of the loss funtion also remains unchanged under the µ-parametrization scaling.

2.2 Mathematical representation of condition 3

Definition 2: The scaling update exponent

Let w_0 be a weight in a neural network of width n at initialisation and w_i its value at i^{th} epoch.
At each epoch we have an update of w which we will denote as follows: $w_{i+1} = w_i - \delta w_i$.
We can demonstrate that following a parametrization: $\forall i \ \frac{||\delta w_i||_2}{||w_i||_2} \underset{n\to\infty}{=} c_i \times n^{k_i} + o(n^{k_i})$ where k_i and c_i are constants independent of n. (This claim is demonstrated in the next sections). We will call k_i the **"scaling update exponent"**.

Condition 3

The condition 3 guarantees having the maximal possible value of the scaling update exponent for every layer and in every training step.
Of course this property is meaningless when there are no constraints on the neural network because without constraints the scaling update exponent is unbounded.
However, in our case, we want to respect also conditions 1 and 2, which means that we want every k_i to be as much close to 0 as possible because otherwise the weights will either blow-up rapidly as n tends to infinity (which opposes conditions 1 and 2), or the weights will not update as much as possible.

The condition 3 implies the interesting property of having weights updating as much as possible while the first two conditions are respected.

In the next sections, we will show how these conditions are equivalent to an optimization problem where the µP is the unique solution.

3 µ-Parametrization Derivation: linear fully connected neural network with three layers

In this section, we will show how we derive our necessary and sufficient optimization problem from the 3 conditions on a simple case: a linear fully connected neural network with three layers and with one-dimensional input and output. We will start by showing how we can get the constraints of our optimization problem from conditions on the expectation and variance of the output of our neural network depending on the choice of the scaling of the different hyperparameters (learning rate and weights initialization). Then we will show how we can derive the objective function from studying the $\mathbf{L}_2$ norm of the weights.

3.1 Notations

We choose to represent our neural network as the following:

- $n \in \mathbb{N}$ is the width of our neural network, ie. the number of neurons in the layers. We will consider that all our layers share the same number of neurons n since we will work on the infinite width limit for all the layers.
- $x \in \mathbb{R}$ is the one dimensional input of our neural network. We choose it to be a deterministic scalar.
- $U \in \mathbb{R}^n$ represents the weights of the first layer. Ux is then the first layer's output. We consider U to be a random matrix with independent Gaussian entries at the initialization.
- $W \in \mathbb{R}^{n \times n}$ represents the weights of the hidden layer. WUx is then the hidden layer's output. We consider W to be a random matrix with independent Gaussian entries at the initialization.
- $V \in \mathbb{R}^n$ represents the weights of the last layer. $VWUx$ is then the output of our neural network. We consider V to be a random matrix with independent Gaussian entries at the initialization.

The parameters of our optimization problem are the following hyperparameters: both the learning rate and the weights initialization scaling depending on the layer. We denote them $\alpha, \beta, \gamma, \eta_\alpha, \eta_\beta, \eta_\gamma$ and they are defined as follows:

- $U_i \sim \mathcal{N}(0, \frac{1}{n^\alpha})$; $W_{ij} \sim \mathcal{N}(0, \frac{1}{n^\beta})$; $V_j \sim \mathcal{N}(0, \frac{1}{n^\gamma})$ at initialization for all $i, j \in [1, n]$.
- $\frac{1}{n^{\eta_\alpha}}$ is the learning rate for the first layer, $\frac{1}{n^{\eta_\beta}}$ is the learning rate for the hidden layer, $\frac{1}{n^{\eta_\gamma}}$ is the learning rate for the last layer.

The standard parametrization (default one in Pytorch) for example is given by: $\alpha = 0$; $\beta = 1$; $\gamma = 0$; $\eta_\alpha = 0$; $\eta_\beta = 0$; $\eta_\gamma = 0$.

3.2 Proof Outline

For our derivation, we will start by showing that every parametrization respecting condition 1, 2 and 3 is necessarily a solution of an optimization problem. (Necessity Proof)

Then we will show that every solution of this optimization problem is respecting conditions 1, 2 and 3. (Sufficiency Proof)

3.2.1 Necessity Proof Outline

- We know that if Condition 1 is respected then necessarily both the expectation and variance of the output converges at every forward pass during training. We derive the sufficient and unique constraints for this implication.
- We know that Condition 3 is respected iff the scaling update exponent is at its maximum. From that, we can derive the objective function to maximize.
- With these constraints and this objective function we have an optimization problem, and every parametrization respecting condition 1, 2 and 3 is a solution to it.

3.2.2 Sufficiency Proof Outline

- We show that with every solution of the optimization problem, the variance of the output converges towards 0 at every forward pass during training.
- Therefore with every solution of the optimization problem, the output converges in distribution towards its expectation at every forward pass.
- Since, we've showed during the necessity proof that every NN respecting the optimization problem has its expectation at every pass converging towards a constant (see section). Therefore, the output is converging in probability towards its expectation. Condition 1 is then respected.
- We show that, if the NN respects the optimization problem, the expectation of the output is a constant not always equal to 0 depending only on the training data (except for the first forward pass). Therefore, the condition 2 is respected.
- Finally, since We know that condition 3 is already respected by maximizing the objective function, the optimization problem is sufficient.

Each one of the solutions of this optimization problem is called a µ-Parametrization.

3.3 Optimization problem derivation: Necessity Proof

3.3.1 Necessary and sufficient constraints derivation from convergence of the expectation

Theorem 1. *At any epoch e, the expectation of the output can be expressed as the following:*

$$\sum_{(k',m',l')\in I_e} \frac{f_{data}(k',m',l',x)}{n^{k'\times\eta_\alpha+l'\times\eta_\beta+m'\times\eta_\gamma+k\alpha+l\beta+m\gamma-f_{sums}(k',m',l')}}$$

Where:

- I_e *is a finite set belonging to* $\{(a,b,c)\in\mathbb{N}^3|\exists k\in\mathbb{N}\ a+b+c=2k+1\}$
- $f_{data}(k',m',l',x)$*, is a function independent from n and of sign independent of k', m' and l'.*
- $\forall(k',m',l')\in I_e\ f_{sums}(k',m',l')\leq\frac{k'+m'+l'+3}{2}$*. The inequality becomes equality when only one of the elements of the tuple is null.*

Proof. See section Proof □

Remark 1: Since f_{data} sign doesn't depend on k', m', and l', the terms of the sum have the same sign. So, no term of the sum can be canceled out by another term. Therefore, if one term of this sum diverges towards infinity than all the sum diverges towards infinity.

Corollary 1.1. *For e fixed, the expectation of the output converges iff* $\forall(k',m',l')\in I_e\ k'\times\eta_\alpha+l'\times\eta_\beta+m'\times\eta_\gamma+k\alpha+l\beta+m\gamma-f_{sums}(k',m',l')\geq 0$

Proof. The proof is trivial considering Remark 1. □

Constraints derivation:
We will now derive the necesseary and sufficient constraints on the hyperparameters for the expectation of the output to converge.

Neccessity of constraints: Let (k',m',l') such that $f_{sums}(k',m',l')=\frac{k'+m'+l'+3}{2}$ be fixed. (It exists according to theorem 1) According to corollary 1, if the expectation of the output con-

verges, then necessarily:

$$
\begin{aligned}
&k' \times \eta_\alpha + l' \times \eta_\beta + m' \times \eta_\gamma + k\alpha + l\beta + m\gamma - f_{sums}(k', m', l') \geq 0 \\
&\iff k' \times \eta_\alpha + l' \times \eta_\beta + m' \times \eta_\gamma + k\alpha + l\beta + m\gamma - \frac{k' + m' + l' + 3}{2} \geq 0 \\
&\iff \alpha + \beta + \gamma - 1.5 + k'(\eta_\alpha + \beta + \gamma - \alpha - 0.5) + l'(\eta_\beta + \alpha + \gamma - \beta - 0.5) + m'(\eta_\gamma + \alpha + \beta - \gamma - 0.5) \geq 0 \\
&\iff \begin{cases} \alpha + \beta + \gamma \geq 1.5 \\ \eta_\alpha + \beta + \gamma - \alpha \geq 0.5 \\ \eta_\beta + \alpha + \gamma - \beta \geq 0.5 \\ \eta_\gamma + \alpha + \beta - \gamma \geq 0.5 \end{cases} \qquad (1)
\end{aligned}
$$

The last equivalence is due to the fact we can always find a large epoch where one element of the tuple (k', m', l') is larger enough compared to the others to make the inequality false.

Sufficiency of constraints: Now, Lets fix any (k',m',l') $\in I_e$, and lets suppose the constraints (1) are respected. Since $f_{sums}(k', m', l') \leq \frac{k'+m'+l'+3}{2}$, by equivalence above:

$$k' \times \eta_\alpha + l' \times \eta_\beta + m' \times \eta_\gamma + k\alpha + l\beta + m\gamma - f_{sums}(k', m', l') \geq 0$$

Therefore, the constraints (1) are necessary and sufficient for having the expectation converging.

3.3.2 Necessary constraints for convergence of variance

Notations: At epoch e, We denote the update of the weights during the backward pass as follows:

- $V^e \leftarrow V^{e-1} - \frac{1}{n^{\eta\alpha}} dV^{e-1}$
- $W^e \leftarrow W^{e-1} - \frac{1}{n^{\eta\beta}} dW^{e-1}$
- $U^e \leftarrow U^{e-1} - \frac{1}{n^{\eta\gamma}} dU^{e-1}$

Theorem 2. *At any epoch e, and for all i, j $\in \mathbb{N}$, we have:*

$$
\begin{cases}
(\frac{||\frac{1}{n^{\eta_\alpha}}(dV^{(e)})_i||_2}{||(V^{(e)})_i||_2})^2 = v_e^2 \times n^{2k_v} + o(n^{2k_v}) \, \textit{Where } k_v = 0.5 - \beta - \gamma - \eta_\alpha + \alpha \\
(\frac{||\frac{1}{n^{\eta_\beta}}(dW^{(e)})_{ij}||_2}{||(W^{(e)})_{ij}||_2})^2 = w_e^2 \times n^{2k_w} + o(n^{2k_w}) \, \textit{Where } k_w = -\beta + \gamma - \eta_\beta + \alpha \\
(\frac{||\frac{1}{n^{\eta_\gamma}}(dU^{(e)})_j||_2}{||(u^{(e)})_j||_2})^2 = u_e^2 \times n^{2k_u} + o(n^{2k_u}) \, \textit{Where } k_u = 0.5 - \beta - \alpha - \eta_\gamma + \gamma
\end{cases}
$$

Proof. See section Proofs □

Theorem 3. *Supposing the expectation of the output converges at any epoch, the variance of the output converges iff $k_v \leq 0, k_w \leq 0$, and $k_u \leq 0$.*

From theorem 3, we derive the other constraints of our optimization problem. Furthermore, k_v, k_w,and k_u represent the scaling update exponents. The objective function to maximize will be the sum of these 3 terms:

$$\begin{aligned} O(\alpha, \beta, \gamma, \eta_\alpha, \eta_\beta, \eta_\gamma) &= -\beta - \gamma - \eta_\alpha + \alpha - \beta + \gamma - \eta_\beta + \alpha - \beta - \alpha - \eta_\gamma + \gamma \\ &= -\alpha - 3\beta - \gamma - 2\eta_\alpha - \eta_\beta - 2\eta_\gamma \end{aligned}$$

Therefore, the optimization problem is the following:

$$\text{maximize} \quad O(\alpha, \beta, \gamma, \eta_\alpha, \eta_\beta, \eta_\gamma)$$

subject to

$$\begin{cases} \alpha + \beta + \gamma \geq 1.5 \\ \eta_\alpha + \beta + \gamma - \alpha \geq 0.5 \\ \eta_\beta + \alpha + \gamma - \beta \geq 0.5 \\ \eta_\gamma + \alpha + \beta - \gamma \geq 0.5 \\ 0.5 - \beta - \gamma - \eta_\alpha + \alpha \leq 0 \\ -\beta + \gamma - \eta_\beta + \alpha \leq 0 \\ 0.5 - \beta - \alpha - \eta_\gamma + \gamma \leq 0 \end{cases}$$

4 References

References

[1] G. Yang. Tensor Programs I: Wide Feedforward or Recurrent Neural Networks of Any Architecture are Gaussian Processes. arXiv preprint arXiv:1910.12478, 2021. `https://arxiv.org/abs/1910.12478`

[2] G. Yang. Tensor Programs II: Neural Tangent Kernel for Any Architecture. arXiv preprint arXiv:2006.14548, 2020. `https://arxiv.org/abs/2006.14548`

[3] G. Yang. Tensor Programs III: Neural Matrix Laws. arXiv preprint arXiv:2009.10685, 2021. `https://arxiv.org/abs/2009.10685`

[4] G. Yang. Tensor Programs IV: Feature Learning in Infinite-Width Neural Networks. arXiv preprint arXiv:2011.14522, 2022. `https://arxiv.org/abs/2011.14522`

[5] G. Yang. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. arXiv preprint arXiv:2203.03466, 2022. `https://arxiv.org/abs/2203.03466`